UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

Department of Computer Science

COS 132 - Imperative Programming
Study Unit 2: Program Structure and Output

Contents

2.1 Introduction

This study unit provides an introduction to programming in C++. In order to get coding as quickly as possible, the study unit introduces an online C++ development environment and an overview of the structure of a C++ program. A traditional first C++ program is presented to illustrate program output. Good programming practices of commenting code and laying code out in a consistent manner are included before an overview of errors that a compiler reports on if the code is not fully accepted by the compiler. Other types of errors do exist, these will be discussed in more detail in later study units.

2.2 Development Environment for C++

The programming development environment you will be using, is the GNU Compiler Collection (GCC). The GCC is a powerful tool that can compile various programming languages, including C++. It is widely used in Linux environments for developing software. The GNU debugger (GDB) will be used for debugging the C++ programs written.

To ease you into programming, the GDB online compiler (OnlineGDB) will be used to begin with. It is a convenient platform for compiling and running C++ programs. This online tool provides an accessible and straightforward way to practice C++ programming without the need for a local setup.

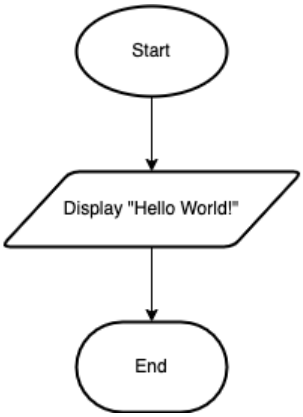
The URL for the GDB online compiler is given by `https://www.onlinegdb.com/online_c++_compiler`. Once you have opened this link in your browser and the website loads, you should be presented with a screen as given in the figure below.

When using OnlineGDB, make sure your set your language to C++. You can type your program into the main windows. Once you are done typing in your program you can press the green *Run* button. This will compile and link your C++ program. Assuming there are no syntax or semantic errors, the program will be run. The program output will be displayed in the console window. If your program does produce errors during compiling and linking, these errors will be displayed in the *stderr* tab (next to the *input* tab) under the main area where you type in your programs.

2.3 Simple C++ Program

Keeping with tradition, the first C++ program you will be writing is *Hello World*. You may have noticed, that this is also the default program loaded by OnlineGDB.

The flowchart for this program is given by:



From the flowchart, you will see that the program has one process. This process will display the words "Hello World!" to the output device. The corresponding C++ program code for this flowchart is given below.

```
#include <iostream>

int main() {
    std::cout<<"Hello World!" << std::endl;
    return 0;
}
```

Listing 2.1: Simple Program

Question 2.1
What is displayed on the screen if the example program is run?

Question 2.2
Modify the example program to display your name. Compile and run your program. Does it behave as expected (i.e., is it semantically correct)?

2.4 Basic C++ Program Structure

All C++ programs require a `main` function. This is the entry point of of the program. The basic structure of a C++ program is as follows:

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main() {
6     //Your code here
```

```
7       return 0;
8   }
```

Listing 2.2: Basic Program Structure

2.4.1 Preprocessor Directives

Preprocessor directives are lines included in the code of your programs that are not actual C++ statements but directives for the preprocessor. These lines are interpreted before the compilation of the program itself begins.

Line 1 in Program listing 2.2 tells the compiler to include the Input/Output stream library. This library is necessary for performing input and output operations, like reading input from the keyboard and writing output to the console (screen). `iostream` stands for input-output stream.

2.4.2 Namespace Declaration

The namespace declaration, line 3 in Listing 2.2, allows for the organisation of code into logical groups and prevents name collisions, especially when your code base includes multiple libraries.
The `using namespace std;` line tells the compiler to use the standard namespace. `std` is the standard namespace that includes features such as I/O operations, for example `cout` and `cin`. The output operation, `cout`, is introduced in Section 2.5, while the input operation is introduced in a later study unit.

2.4.3 Main Function

All C++ programs need a `main` function. The execution of a C++ program starts from the main function definition (from line 5) and ends at the trailing } in line 8. `int` before `main` function indicates that main will return an integer value. This return value is used by the operating system to determine how the program exited (0 indicates success, refer to line 7)

2.4.3.1 Body of the Main Function

The body of the main function is the code between the curly braces. That is, { on line 5 and } on line 8 of Listing 2.2.

The body of the main function will include one or more program statements.

2.4.3.2 Program Statements

A program statement is an instruction/action that the computer needs to carry out (execute). Statements are specified sequentially and are delimited (separated) by a semi-colon (‘;’).

In Listing 2.1, std :: cout<<”Hello␣World!”;[1] is the first program statement and **return** 0;

[1]Note the use of the ␣ character to indicate the presence of whitespace in output.

is the second. The program in Listing 2.1 will output "`Hello, World!`" to the console.

return 0; is a special type of program statement.

Programe statements may be grouped together in a statement block. A statement block is defined within curly braces. The body of the main function is therefore considered a special type of statement block.

2.4.3.3 Return Statement

The return statement, line 7 of Listing 2.1, in the `main` function indicates the exit status of the program. Returning 0 signifies that the program ended successfully.

2.4.3.4 Program Comments

Program comments are indicated by `//` if the comment is to be specified on a single line. For example, line 6 in Listing 2.1.

Comments can also span multiple lines by using placing the comment between the characters `/*` and `*/`. For example:

```
/*
 This comment can be longer
 You don't need to add // each time
 This is useful for longer explanations
*/
```

Comment statements are ignored by the compiler and therefore are seen by many as being merely there for "cosmetic" reasons. A good programming practice is to give a short description at the beginning of a program or function to tell what the code actually does.

2.4.3.5 Keywords

Keywords in C++ are reserved words. This means they cannot be used in a program in any way, for example as variables or function names, other than for the reason they are defined in C++. Examples of keywords in Listing 2.2 are: `using`, `namespace` and `return`. Note, `include` is considered a preprocessor keyword. `std` is a keyword that relates too namespaces, and `main` is reserved for the function that executes first in C++. A list of C++ keywords are available from `https://en.cppreference.com/w/cpp/keyword`.

2.5 Output

In C++, output is handled using the `iostream` library. This library should be included as a preprocessor directive whenever you want your program to be able to handle input or output.

```
#include <iostream>
```

The library provides the definitions for the `<<` operator as well defintions for the `cout` and `endl` operations shown in Listing 2.1. Operator `<<`, indicates that whatever follows it is to be sent (piped) to the defined output channel. The output channel in this case is the console, hence the use of `cout` before the operator. `endl` is used to insert a newline character (similar to pressing Enter) into the output. Furthermore, it flushes the output buffer. This means that all characters that have been piped for output will be pushed to the output device (the console in this case).

Both `cout` and `endl` are defined in the `std` namespace. If the namespace declaration is not specified prior to the main function, the scoping operator (`::`) needs to be used to tell the compiler where `cout` and `endl` are defined. If the namespace declaration given by line 3 of Listing 2.2 was included in program for Listing 2.1, the line that writes "Hello World!" to the console, the statement would be writtenas follows: cout << "Hello,␣World!"<< endl;.

Question 2.3
Will the following program compile? If not, what seems to be the problem?

```
#include <iostream>
using namespace std;

int main() {
    cout >> "COS132␣2024";
    return 0;
}
```

Question 2.4
Write a program to display the following figure on the screen.

```
        *
      * * *
    * * * * *
  * * * * * * *
* * * * * * * * *
        |
        |
```

(HINT: You must use a number of `cout` statements one after the other)

Question 2.5
Compile and run the program given in Listing 2.3.

```
#include <iostream>

using namespace std;

int main() {
    cout << "Hello" << endl;
    cout << endl;
    cout << "Joan" << endl;
    return 0;
}
```

Listing 2.3: Basic Output Example

Now remove the `cout << endl;` statement and run the program again. How does the output change?

2.6 Commenting and Indenting Code

Style is a crucial component of professionalism in software development. Clean code that follows stylistic conventions is easier to read, maintain, and share with colleagues. When a consistent style is used throughout a project, it makes it easier for the developers working on the project to understand each other's code.

Two aspects of clean code include commenting code and indenting code. The sections that follow will highlight the practices for commenting code and the rules for code layout, of which indenting code is an aspect of.

2.6.1 Commenting practices

Comments are included in code to clarify code and give additional information that cannot be included in the code. The principle is rather to write *self documenting code* than to over comment. It is important to realise that comments cannot rectify bad code. If code is bad, it needs to be rewritten, not commented.

Some of the main practices for writing good comments in code are:

- Every file containing code must start with a comment containing the name(s) and student numbers of the author(s), the date of last edit as well as the purpose of the file. Use the proper tags for author and date as specified by the documentation generator[2] of your choice.
- Avoid redundancy and duplication of what is already clear in the code.
- Make sure comments and code agree. Often programmers change code without updating the accompanying comments. This is unacceptable. Wrong comments is worse than no comments.
- Use a formal writing style to state facts in full sentences that are concise and to the point. Writing concise explanations is often trickier than writing code!
- Every function definition (remember this practice for when we consider functions) must be preceded by a comment that briefly describes what the function does. Use the proper tags defined by the documentation generator of your choice to describe all parameters and the return value of the function

It is important to note that comments are also used to enhance the clarity of automatically generated documentation. For this reason comments that are embedded in the code should follow the syntax specified by the chosen documentation generator.

[2] Javadoc can be used when writing Java code and a package like Doxygen is suitable for C and C++ code

2.6.2 Layout rules

Good and consistent code layout enhances code readability. If code is readable, it is inherently more understandable and consequently reliable and maintainable. Below are some pointers on layout that need to be taken into account a promote code readability. Many of these pointers will become more apparent as we progress through the study units.

- Use indentation and blank lines to reveal the subordinate nature of blocks of code. Each line which is part of the body of a control structure (if, while, dowhile, for, switch) is indented one tab stop from the margin of its controlling line. The same rule applies to function, struct, or union definitions, and aggregate initialisers.
- Use blank lines freely to separate parts of a function or method that are logically distinct.
- Use a blank space around binary operations.
- Leave a blank space after (and not before) each comma, colon or semicolon.
- Every line must fit 80 columns.
- We do not require specific placement of opening and closing braces. We do, however, require consistency. None of the styles are difficult to understand, choose one of them and use it consistently throughout all the code of one project.

```
int main()
{
        cout << "Programming is great fun!" << endl;
        return 0;
}
```

Listing 2.4: Example style 1

```
int main(){
        cout << "Programming is great fun!" << endl;
        return 0;
}
```

Listing 2.5: Example style 2

```
int main()
        {
                cout << "Programming is great fun!" << endl;
                return 0;
        }
```

Listing 2.6: Example style 3

- If a hard copy of a program list is printed, insert page-breaks to avoid code blocks to flow onto a next page when printed.

2.7 Errors when coding

Different types of errors exist when coding. Errors are placed in two broad categories, errors that take place during compiling and linking and errors that occur when the program is executing. A program that does not compile or link cannot run. A program that successfully compiles and links is not necessarily error free.

In this section we will consider the syntax error and the semantic error, both are examples of errors that occur during compilation.

2.7.1 Syntax errors

The first type of error you will encounter when coding are syntax errors. These occur when the C++ compiler is unable to parse the code because the code violates the expected code structure. For example, incorrect spelling of a keyword and missing a semi-colon will result in a syntax error. The compiler will try and resolve the errors to the best of its ability. The line where the error occurred may therefore not be at the location as indicated by the compiler error messages.

Question 2.6
Identify all the syntax errors in the following C++ program.

```
#includ <iostream>

using namespace std

int main() {
    cout >> "COS132 2024";
    return 0;
```

2.7.2 Semantic errors

Semantic errors occur when the statement written in the program is not meaningful to the compiler. For example, assuming you want to add the values `a` and `b` and place the result in `c`. The statement a + b = c; would result in a semantic error. The compiler is expecting a statement with `c` on the left, that is c = a + b.

2.8 Document Change Log

Date	Change
25 Feb 2024	Initial preparation and presentation of the document.